

SOURCE LOG: src\chroma_core\layers_clean.py

```
#----- 1: RMSNorm -----
"""
Z:\flowch\src\chroma_core\layers_clean.py
Chroma v50.
MMDiT.
"""

import torch
import torch.nn as nn
import math

class ChromaTelemetry:
    @staticmethod
    def verify(tensor: torch.Tensor, name: str, expected_dim: int = None):
        """ Autograd."""
        if not tensor.is_cuda:
            print(f"[{name}]: CUDA RAM!")
        if expected_dim and len(tensor.shape) != expected_dim:
            raise ValueError(f"[{name}]: {expected_dim}D, {len(tensor.shape)}D")
        if torch.isnan(tensor).any():
            print(f"[{name}]: NaN!")

class RMSNorm(nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        # 
        self.scale = nn.Parameter(torch.ones(dim))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # 
        ChromaTelemetry.verify(x, "RMSNorm.input")
        # 
        variance = x.pow(2).mean(-1, keepdim=True)
        # (BF16)
        out = x * torch.rsqrt(variance + self.eps)
        return out * self.scale.to(x.dtype)

#----- 1 -----
#----- 2: MLPEmbedder -----
def timestep_embedding(timesteps: torch.Tensor, dim: int, max_period: int = 10000) -> torch.Tensor:
    """
    (Rectified Flow).
    1000.0 
    [1.3].
    """
    ChromaTelemetry.verify(timesteps, "timestep_embedding.input")

    half_dim = dim // 2
    frequencies = torch.exp(
        -math.log(max_period) * torch.arange(start=0, end=half_dim, dtype=torch.float32, device=timesteps.device) / half_dim
    )

    # 
    args = timesteps.to(torch.float32).unsqueeze(-1) * frequencies.unsqueeze(0)
    embedding = torch.cat([torch.sin(args), torch.cos(args)], dim=-1)

    # 
    if dim % 2 == 1:
        embedding = torch.nn.functional.pad(embedding, (0, 1, 0, 0))

    return embedding.to(torch.bfloat16)

class MLPEmbedder(nn.Module):
    def __init__(self, in_dim: int, out_dim: int):
        super().__init__()
        # MLP 
        self.in_proj = nn.Linear(in_dim, out_dim)
```

```

self.silu = nn.SiLU()
self.out_proj = nn.Linear(out_dim, out_dim)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    ChromaTelemetry.verify(x, "MLPEmbedder.input")
    # ██████████ ██████ ████████████████████ █ BF16 ██████ ██████████
    h = self.silu(self.in_proj(x.to(torch.bfloat16)))
    out = self.out_proj(h)
    ChromaTelemetry.verify(out, "MLPEmbedder.output")
    return out

#-----██████████ ██████ █2 -----
#-----█████ █3: ████████████████████ distribute_modulations -----
class ChromaModulationBus(nn.Module):
    def __init__(self, hidden_size: int, num_double: int = 19, num_single: int = 38):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_double = num_double
        self.num_single = num_single

        # ██████ ████████████████████████████████████████:
        # DoubleStream: 6 ████████████████████████ (shift/scale/gate ███ txt █ img ██████████)
        # SingleStream: 3 ████████████████████████ (shift/scale/gate ███ ████████████████████████████████████████)
        self.total_mod_chunks = (num_double * 6) + (num_single * 3)
        self.expected_features = self.total_mod_chunks * hidden_size

    def distribute_modulations(self, monolithic_tensor: torch.Tensor) -> dict:
        """
        ██████████ ████████████████████████████████████████ ████████████████████████████████████████ ████████████████████████████████████████
        ███ ████████████████████████████████████████ ████████████████████████████████████████ ████████████████████████████████████████ 57 ████████████████████████████████████████ [1.3, 1.4].
        """
        ChromaTelemetry.verify(monolithic_tensor, "ModulationBus.input")
        batch_size = monolithic_tensor.shape[0]

        if monolithic_tensor.shape[-1] != self.expected_features:
            raise ValueError(
                f"█ ████████████████████████████████████████ ████████████████████████████████████████ {monolithic_tensor.shape[-1]} ████████████████████████████████████████, "
                f"███████████████████████████████████████ {self.expected_features}"
            )

        # ██████████ split ███ ███ ████████████████████████████████████████ ████████████████████████████████████████ hidden_size
        chunks = torch.split(monolithic_tensor, self.hidden_size, dim=-1)
        chunk_idx = 0

        mod_map = {
            "double": [],
            "single": []
        }

        # ██████████ ██████ 19 ██████████ DoubleStream (6 ████████████████████████████████████████)
        for i in range(self.num_double):
            block_mods = {
                "txt_shift": chunks[chunk_idx],
                "txt_scale": chunks[chunk_idx + 1],
                "txt_gate": chunks[chunk_idx + 2],
                "img_shift": chunks[chunk_idx + 3],
                "img_scale": chunks[chunk_idx + 4],
                "img_gate": chunks[chunk_idx + 5]
            }
            mod_map["double"].append(block_mods)
            chunk_idx += 6

        # ██████████ ██████ 38 ██████████ SingleStream (3 ████████████████████████████████████████)
        for i in range(self.num_single):
            block_mods = {
                "shift": chunks[chunk_idx],
                "scale": chunks[chunk_idx + 1],
                "gate": chunks[chunk_idx + 2]
            }
            mod_map["single"].append(block_mods)
            chunk_idx += 3

```

```
return mod_map

#----- 03 -----
#----- 04: NerfEmbedder LRU-----
class NerfEmbedder(nn.Module):
    def __init__(self, in_channels: int = 3, num_frequencies: int = 64):
        super().__init__()
        self.in_channels = in_channels
        self.num_frequencies = num_frequencies

        # LRU- [1.4]
        self._cache = {}
        self._max_cache_size = 8

    def _get_frequency_grid(self, device: torch.device) -> torch.Tensor:
        cache_key = f"freq_{device}"
        if cache_key in self._cache:
            return self._cache[cache_key]

        # float32
        frequencies = torch.logspace(
            start=0.0,
            end=math.log2(10000.0),
            steps=self.num_frequencies,
            base=2.0,
            dtype=torch.float32,
            device=device
        )

        # , 
        if len(self._cache) >= self._max_cache_size:
            self._cache.pop(next(iter(self._cache)))
        self._cache[cache_key] = frequencies
        return frequencies

    def forward(self, coords: torch.Tensor) -> torch.Tensor:
        # CUDA - FP8 [1.4]
        with torch.amp.autocast('cuda', enabled=False):
            pass
            # float32
            x_f32 = coords.to(torch.float32)
            ChromaTelemetry.verify(x_f32, "NerfEmbedder.input_f32")

            freq_grid = self._get_frequency_grid(x_f32.device)

            # 
            # coords: [B, N, in_channels] -> [B, N, in_channels, num_frequencies]
            scaled = x_f32.unsqueeze(-1) * freq_grid.view(1, 1, 1, -1)

            # /
            sin_feat = torch.sin(scaled)
            cos_feat = torch.cos(scaled)

            # 
            out = torch.cat([sin_feat, cos_feat], dim=-1).flatten(start_dim=-2)
            ChromaTelemetry.verify(out, "NerfEmbedder.output_f32")

            # float32, Autograd- [1.3]
            return out

#----- 04 -----
#----- 05-7: DoubleStreamBlock SingleStreamBlock -----

class DoubleStreamBlock(nn.Module):
    def __init__(self, hidden_size: int, num_heads: int = 24, head_dim: int = 128):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_heads = num_heads
        self.head_dim = head_dim

        self.txt_norm = RMSNorm(hidden_size)
        self.img_norm = RMSNorm(hidden_size)
```

```

self.txt_attn = nn.Linear(hidden_size, hidden_size * 3)
self.img_attn = nn.Linear(hidden_size, hidden_size * 3)

self.query_norm = RMSNorm(head_dim)
self.key_norm = RMSNorm(head_dim)

self.txt_proj = nn.Linear(hidden_size, hidden_size)
self.img_proj = nn.Linear(hidden_size, hidden_size)

def forward(self, txt: torch.Tensor, img: torch.Tensor, mods: dict) -> tuple[torch.Tensor, torch.Tensor]:
    ChromaTelemetry.verify(txt, "DoubleStream.txt_in", 3)
    ChromaTelemetry.verify(img, "DoubleStream.img_in", 3)

    # ■■■■■■■■■■ ■■■■■■■■■■ ■ BF16
    txt_mod = self.txt_norm(txt) * (1 + mods["txt_scale"].to(txt.dtype)) + mods["txt_shift"].to(txt.dtype)
    img_mod = self.img_norm(img) * (1 + mods["img_scale"].to(img.dtype)) + mods["img_shift"].to(img.dtype)

    # ■■■■■■■■■■ ■■■■: ■■■■■■■■■■ ■■■■■■■■■■ ■ ■■■■ ■■■■■■ (■■■■■■■■■■■ ■■■■■■ mat1/mat2)
    txt_mod = txt_mod.to(self.txt_attn.weight.dtype)
    img_mod = img_mod.to(self.img_attn.weight.dtype)

    # ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ QKV
    qkv_txt = self.txt_attn(txt_mod)
    qkv_img = self.img_attn(img_mod)

    # ■■■■■■■■■■ ■■■■■■■■■■ ■ ■■■■■■■■ ■■ ■■■■ ■■■■■■ ■■■■■■■■■■ ■■■■■■
    txt_out = self.txt_proj(qkv_txt[..., :self.hidden_size].to(self.txt_proj.weight.dtype)) * mods["txt_gate"].to(txt.dtype)
    img_out = self.img_proj(qkv_img[..., :self.hidden_size].to(self.img_proj.weight.dtype)) * mods["img_gate"].to(img.dtype)

    return txt + txt_out.to(txt.dtype), img + img_out.to(img.dtype)
#-----■■■■■■■■■■■ ■■■■■■ ■7 -----

#-----■■■■■ ■9: ■■■■■■■■■■ SingleStreamBlock ■ ■■■■■■■■ ■■■■■■■■■■ -----
class SingleStreamBlock(nn.Module):
    def __init__(self, hidden_size: int, mlp_hidden_dim: int = 12288):
        super().__init__()
        self.hidden_size = hidden_size
        self.norm = RMSNorm(hidden_size)
        self.linear1 = nn.Linear(hidden_size, hidden_size * 3 + mlp_hidden_dim)
        self.linear2 = nn.Linear(mlp_hidden_dim, hidden_size)

    def forward(self, x: torch.Tensor, mods: dict) -> torch.Tensor:
        ChromaTelemetry.verify(x, "SingleStream.input", 3)

        # ■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■ ■■■■■■
        x_mod = self.norm(x) * (1 + mods["scale"].to(x.dtype)) + mods["shift"].to(x.dtype)

        # ■■■■■■■■■■ ■■■■ ■■■■■■■■■■ ■■■■■■: ■■■■■■■■■■ ■■■■■■ ■ ■■■■■■ ■■■■■■ (BF16 -> FP32 / FP8)
        x_mod = x_mod.to(self.linear1.weight.dtype)

        # ■■■■■■ ■■■■■■ ■■■■■■■■■■ ■■■■■■ ■ ■■■■■■■■
        linear1_out = self.linear1(x_mod)
        gate_gate = mods["gate"].to(x.dtype)

        # ■■■■■■■■■■ MLP-■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■ ■■■■■■■■■■
        mlp_in = linear1_out[..., (self.hidden_size * 3):]
        mlp_out = self.linear2(torch.nn.functional.silu(mlp_in))

        out = mlp_out.to(x.dtype) * gate_gate
        return x + out
#-----■■■■■■■■■■■ ■■■■■■ ■9 -----

#-----■■■■■■■■■■■ ■■■■■■ ■5 -----
#-----■■■■■ ■6: ■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■ ■■■■■■ (Sandbox-Run) -----
def run_cold_reactor_test():
    """
    ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■ ■■ ■■■■■■ ■■■■■■.
    ■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■, ■■■■■■■■■■ RMSNorm, ■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■
    ■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■.
    """

```

